Microsoft’s Brainwave Project

Sparsh Mittal

Microsoft's Brainwave Architecture: Introduction and Software Stack

Sparsh Mittal

Courtesy: Eric Chung et al.

Images are courtesy of: http://learningsys.org/nips17/assets/slides/brainwave-nips17.pdf

Acronyms

- DNN – Deep neural network
- DPU – DNN processing unit
- MLP – Multilayer perceptron
- LSTM – Long-short term memory
- GRU – Gated recurrent unit
- FPGA – Field-programmable gate array
- DRAM – Dynamic random-access memory
- HW – Hardware
- NPU – Neural processing unit
- ISA – Instruction set architecture
- ML – Machine learning
- FU – Functional unit
- NIC – Network interface card

ASIC vs. FPGA

- An ASIC is a fixed function device, and it can take years to design an updated chip.
- By comparison, a new soft DPU (e.g., better suited to a more powerful FPGA) can be designed in few days to weeks.
  - Example: Bing has real-time latency requirements and its algorithms/models evolve rapidly.
  - Microsoft engineers regularly adapt Brainwave NPU ISA and microarchitecture to new requirements, pushing modified FPGA images into production environments within days to weeks.

Flexibility of FPGAs

- FPGAs are ideal for adapting to rapidly evolving ML
- FPGAs can efficiently implement CNNs, LSTMs, MLPs, reinforcement learning, feature extraction, decision trees, and so on.
- FPGAs facilitate inference-optimized numerical precision. This allows user to the change precision and function implemented
- Exploit sparsity and deep compression for larger and faster models

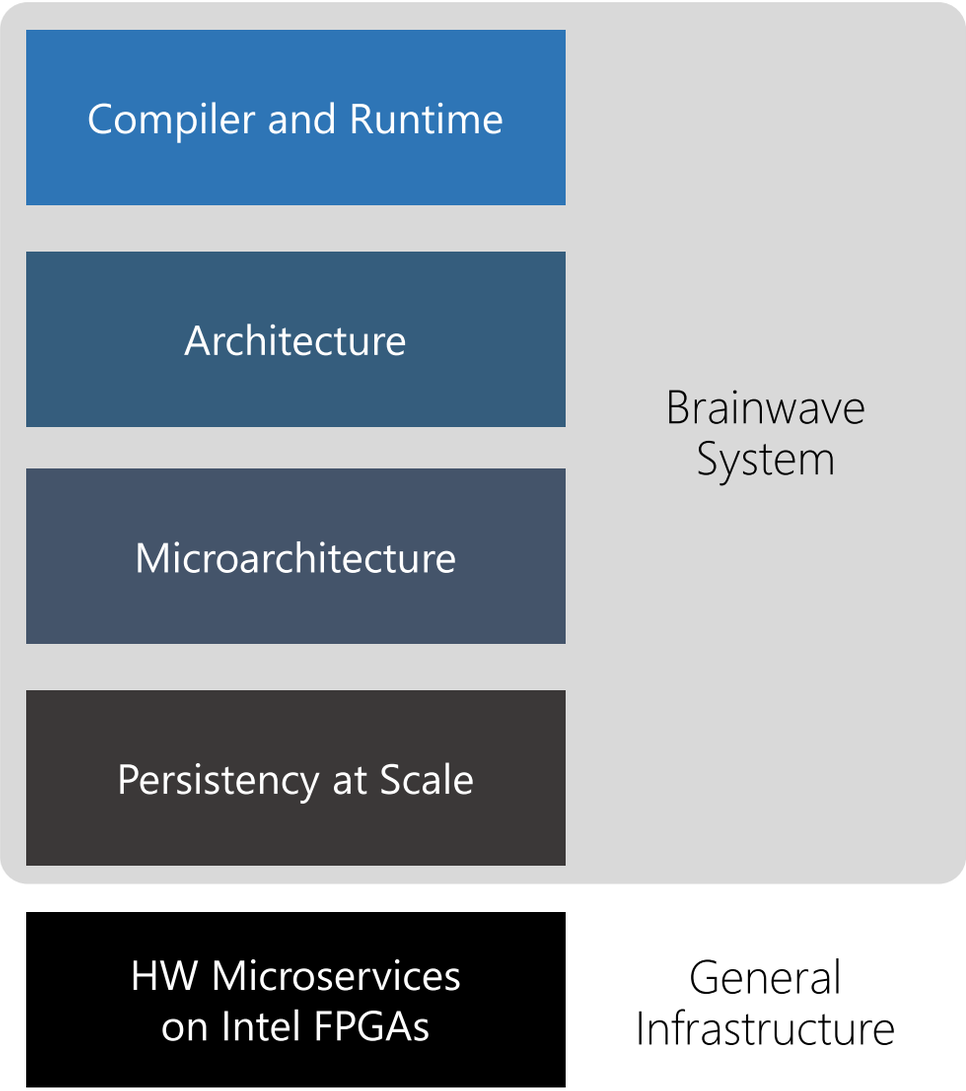
Performance of FPGA

- Ultralow latency
- Even in worst case scenario, a large LSTM-based NLP model takes only tens of milliseconds for inference on an FPGA. This latency is negligible compared to end-to-end latency

NLP = Natural Language Processing

Software Stack

How It Works: The Brainwave Stack



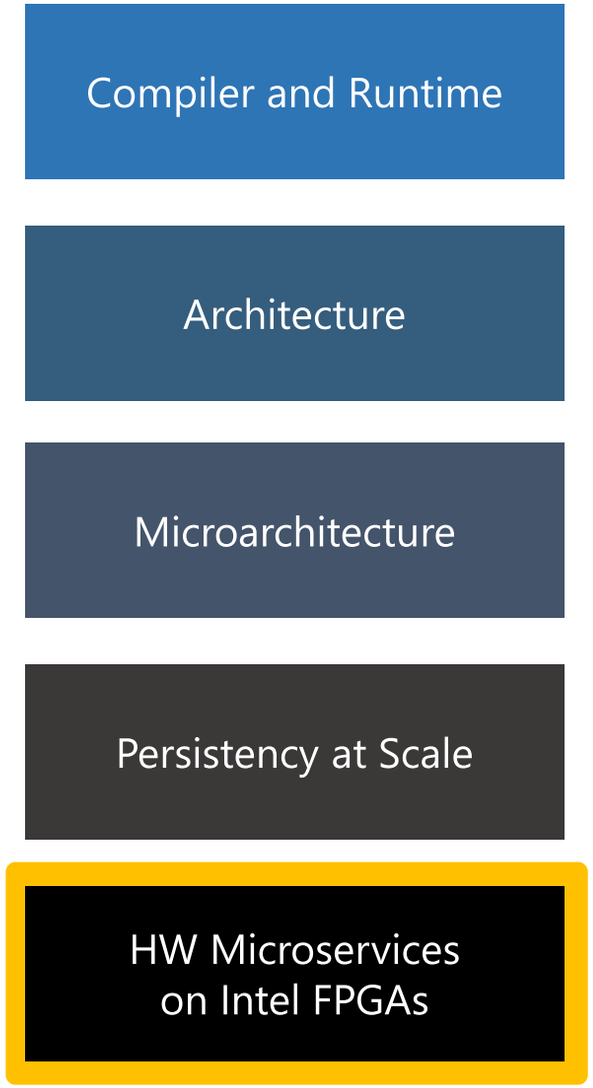
http://learningsys.org/nips17/assets/slides/brainwave-nips17.pdf

How It Works: The Brainwave Stack

Compiler and Runtime	A framework-neutral federated compiler and runtime for compiling pretrained DNN models to soft DPUs
Architecture	Adaptive ISA for narrow precision DNN inference Flexible and extensible to support fast-changing AI algorithms
Microarchitecture	Brainwave Soft DPU microarchitecture Highly optimized for narrow precision and low batch
Persistency at Scale	Persist model parameters entirely in FPGA on-chip memories Support large models by scaling across many FPGAs
HW Microservices on Intel FPGAs	Intel FPGAs deployed at scale with HW microservices

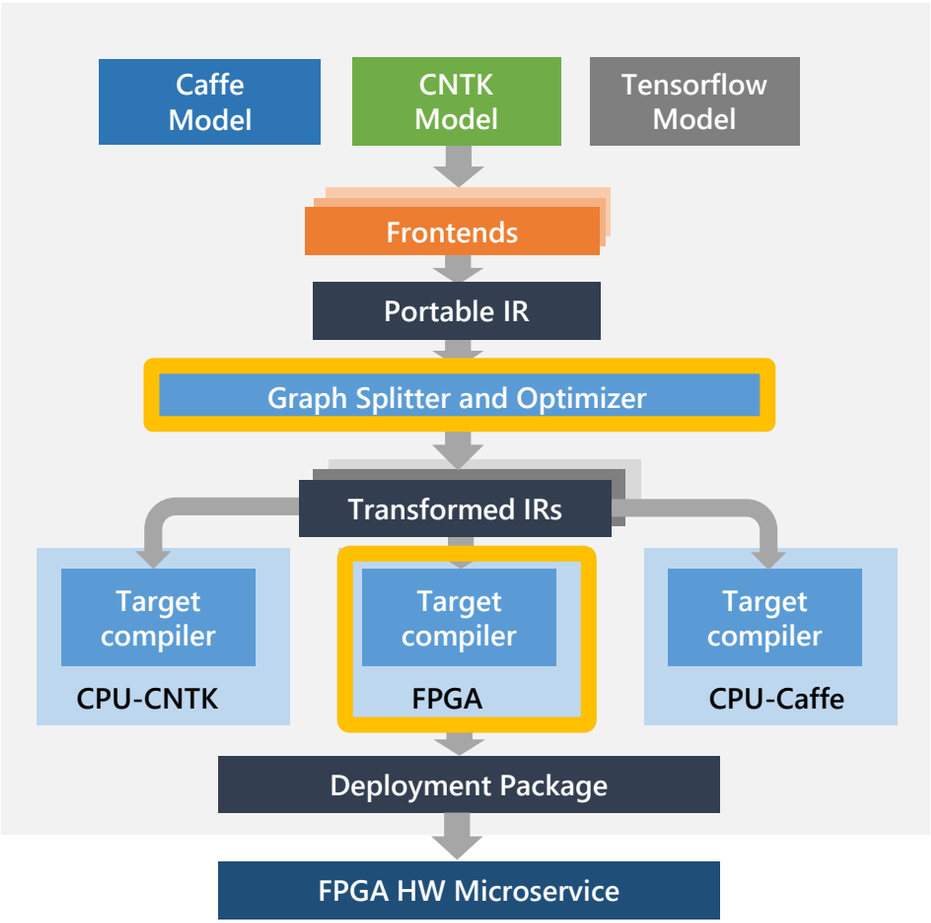
The Brainwave Stack

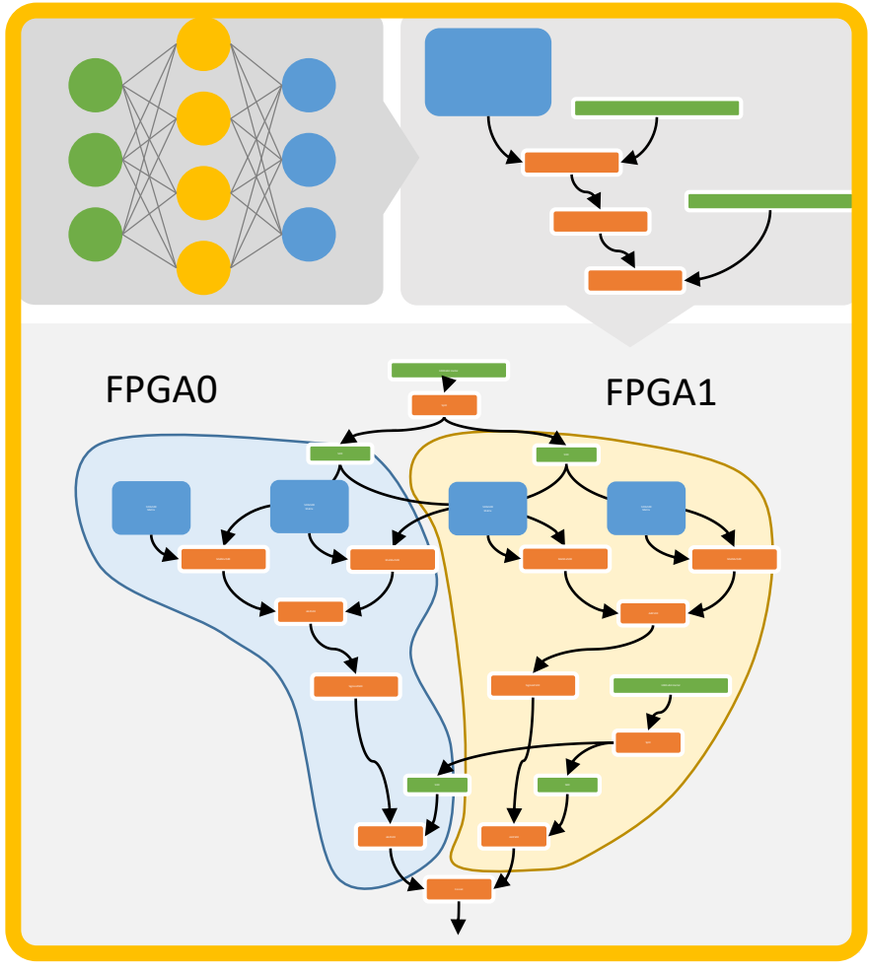
- Brainwave takes DNNs and compiles them so that they can be run on the Brainwave soft DPU
- Soft DPU is loaded onto FPGA
- These FPGAs are placed in Microsoft servers



http://learningsys.org/nips17/assets/slides/brainwave-nips17.pdf

Brainwave Compiler and Runtime





http://learningsys.org/nips17/assets/slides/brainwave-nips17.pdf

Brainwave Compiler and Runtime

- During offline compilation, Brainwave tool flow splits DNN models into subgraphs, each of which can fit into on-chip FPGA memory or run on the CPU
- When subgraphs are pinned to an FPGA, many terabytes per seconds of memory bandwidth can be delivered to a single-DNN query execution
- This enables FPGA to achieve ultralow latency execution at near-peak efficiency—up to 90% utilization at batch 1

- Partitioned subgraphs are passed down to device-specific backend tools.
- Each backend may map subgraphs into optimized libraries (e.g., AVX-tuned BLAS kernels on x86 CPUs) or compile optimized assembly automatically.

BLAS—basic linear algebra subprograms

If on-chip memory of one FPGA is exhausted

- If on-chip memory of one FPGA is exhausted, the system user can allocate more FPGAs for pinning remaining parameters by scaling up a single hardware microservice and leveraging the elasticity of cloud-scale resources.
- With other single-chip NPUs (with no direct connectivity to other NPUs), one would need scaling down models undesirably to fit into limited single-device on-chip memory or would require spilling of models into off-chip DRAM, increasing latencies by an order of magnitude or more.

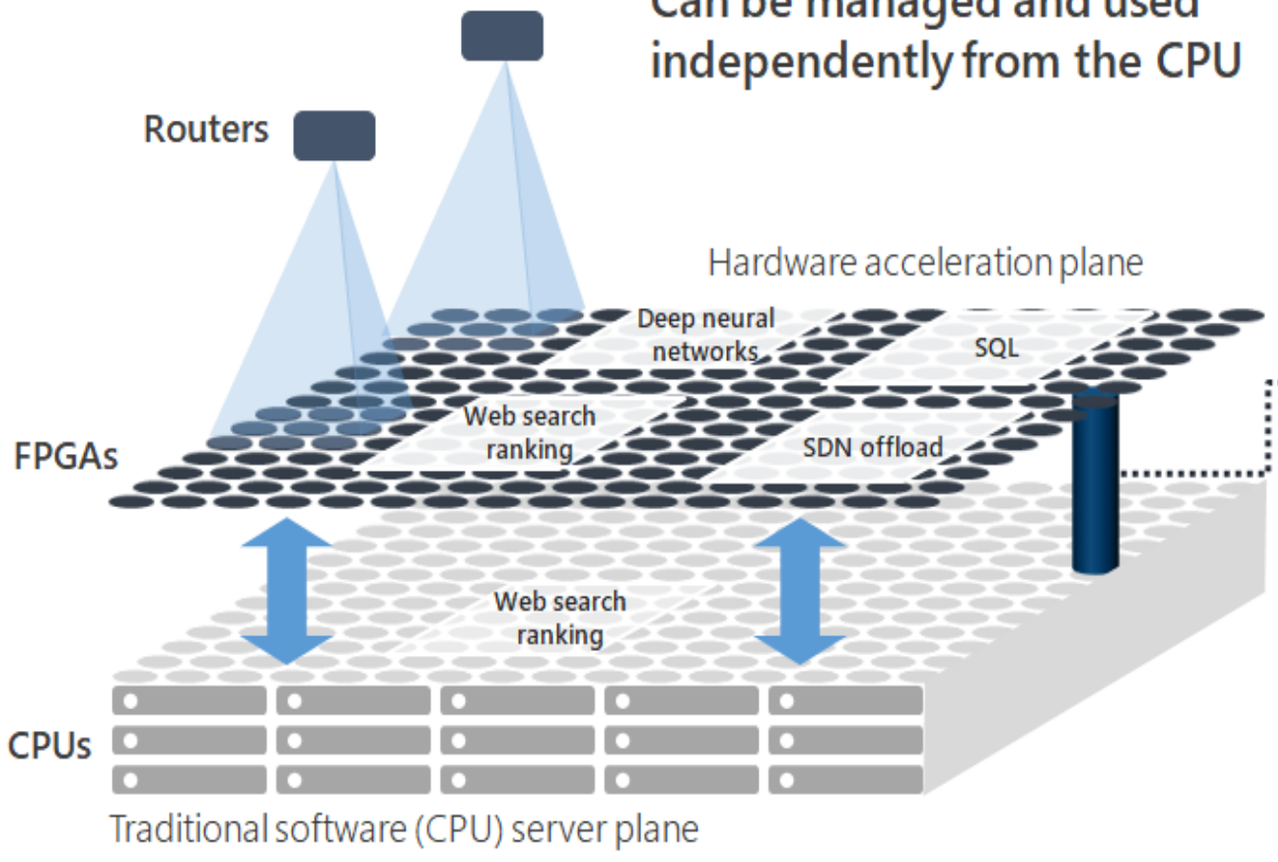
Benefits of Pooling FPGAs

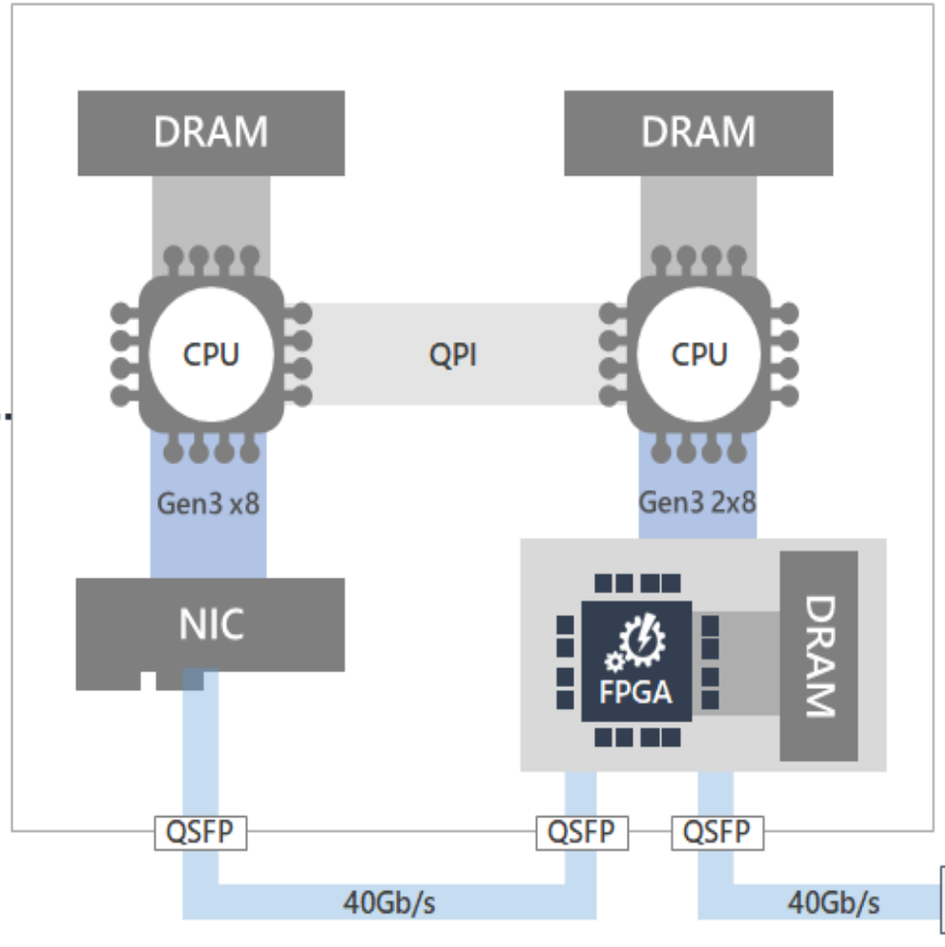
- If a workload does not fit in one FPGA, it can be spread across multiple FPGAs (hop latency ~2 ms).
  - That too, completely independently of the underlying CPUs in the servers.
- With FPGAs linking to Azure network directly, any processor in Azure can access any single FPGA or cluster of them running a DNN for inference.

Hardware Microservices on FPGAs

Interconnected FPGAs form a separate plane of computation

Can be managed and used independently from the CPU





Challenges in Pooling FPGAs

- Issue: when multiple FPGAs are allocated to host a single model, aggregate FPGA capacity (e.g., sustainable queries per second) within a single hardware microservice tends to exceed the traffic that a single CPU client can drive.
- Idea: many CPU clients are assigned to share an instance of a hardware microservice running on a set of FPGAs, better matching the compute ratio between CPU and FPGAs.

Use of CPU

- Operators that are unsupported or not profitable for offload are grouped into subgraphs assigned to CPUs.
- This heterogeneous approach preserves operator coverage by leveraging CPUs as a catch—all for currently unsupported or rarely exercised operators.